Fundamentals of Distributed Systems
Assignment (Q1)G25AI1058

Q1. Resilient State Machine Replication: Paxos & PBFT

Objective: -To design, implement, and rigorously test a highly resilient distributed state machine capable of enduring both benign (crash) faults and malicious (Byzantine) faults. This assignment demands practical integration of **Leader Election, Paxos, Practical Byzantine Fault Tolerance (PBFT),** and **Process Coordination** within a simulated, unstable network environment.

Name: Harshita Gaur **Roll Number: G25AI1058**

Link to GitHub Repository: https://github.com/g25ai1058/distributed-consensus-engine

---

Document Report

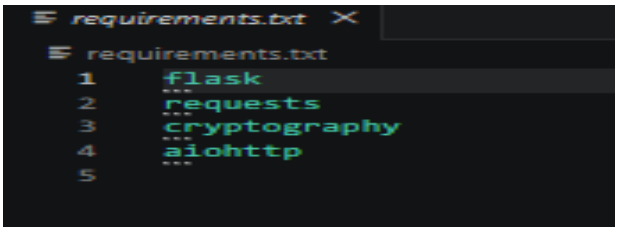
Step 1: - Create Project Folder-distributed-consensus-engine

- mkdir distributed-consensus-engine
- cd distributed-consensus-engine

Step 2: Install Required Packages

- nano requirements.txt

  flask
  requests
  cryptography
  aiohttp (save it)



- pip install -r requirements.txt

1. **Process Coordination & Leader Election**
   - **Process coordination** refers to the techniques used to manage interactions among distributed processes. Since processes may execute concurrently and share resources, coordination helps maintain consistency, prevent conflicts, and ensure reliable system operation. Common coordination tasks include resource allocation, synchronization, task scheduling, and fault handling.
   - **Leader election** is the process of selecting one node as the coordinator (leader) among a group of distributed nodes. The leader manages tasks such as:

- Coordinating communication
- Managing shared resources

- Handling consensus decisions
- Detecting and recovering from failures
  When the current leader fails, a new leader must be elected.

❖ **create one folder inside distributed-consensus-engine: -** mkdir src

❖ **create file inside src folder**:-touch Leader_Election.py (write the code in it):-

```
import threading
import time

NUM_NODES = 5

class Node:
    def __init__(self, node_id):
        self.node_id = node_id
        self.is_leader = False
        self.alive = True
        self.last_heartbeat = time.time()

nodes = [Node(i) for i in range(1, NUM_NODES + 1)]

# Highest ID becomes initial leader
leader = max(nodes, key=lambda n: n.node_id)
leader.is_leader = True

print("\n=== Initial Leader Election ===")
for node in nodes:
    role = "Leader" if node.is_leader else "Follower"
    print(f"Node {node.node_id} -> {role}")

def send_heartbeats():
    global leader

    while True:
        if leader and leader.alive:
            for node in nodes:
                if node.node_id != leader.node_id:
                    node.last_heartbeat = time.time()

            print(f"[Heartbeat] Leader Node {leader.node_id}")
            time.sleep(2)
        else:
            break

def monitor_nodes():
    global leader

    while True:
        time.sleep(3)

        if leader and not leader.alive:

            print(
                f"\n[Failure Detected] Leader Node {leader.node_id} crashed!"
            )

            candidates = [
                n for n in nodes
                if n.alive and n.node_id != leader.node_id
            ]

            if candidates:
                new_leader = max(
                    candidates,
                    key=lambda n: n.node_id
                )

                for node in nodes:
                    node.is_leader = False

                new_leader.is_leader = True
                leader = new_leader

                print(
                    f"[Election] New Leader Elected -> Node {leader.node_id}\n"
                )

                threading.Thread(
                    target=send_heartbeats,
                    daemon=True
                ).start()

                break

# Start heartbeat thread
threading.Thread(
    target=send_heartbeats,
    daemon=True
).start()

# Start monitoring thread
threading.Thread(
    target=monitor_nodes,
    daemon=True
).start()

# Simulate leader crash after 10 sec
time.sleep(10)

print("\n***** Simulating Leader Crash *****")
leader.alive = False

# Keep program running
while True:
    time.sleep(1)

print("\n***** Leader Crashed *****\n")
time.sleep(2)

print("Node 5: Leader failure detected!\n")
time.sleep(1)

# Elect new leader
leader = 4
print(f"New Leader Elected: Node {leader}\n")

# New leader heartbeats
while True:
    print(f"Leader {leader} heartbeat")
    time.sleep(1)
print("Transaction Committed")
```

- The code is a **simple simulation of Leader Election in a distributed system**. It demonstrates what happens when a leader node fails and another node is elected as the new leader.

❖ **Run command:-**python Leader_Election.py The output provides that, in case of leader failure, the remaining active nodes elect a new leader automatically.

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/Assignment/distributed-consensus-engine/src$ python Leader_Election.py

=== Initial Leader Election ===
Node 1 -> Follower
Node 2 -> Follower
Node 3 -> Follower
Node 4 -> Follower
Node 5 -> Leader
[Heartbeat] Leader Node 5
[Heartbeat] Leader Node 5
[Heartbeat] Leader Node 5
[Heartbeat] Leader Node 5
[Heartbeat] Leader Node 5

***** Simulating Leader Crash *****

[Failure Detected] Leader Node 5 crashed!
[Election] New Leader Elected -> Node 4

[Heartbeat] Leader Node 4
[Heartbeat] Leader Node 4
[Heartbeat] Leader Node 4
[Heartbeat] Leader Node 4
[Heartbeat] Leader Node 4
[Heartbeat] Leader Node 4
```

Initial Leader Election and Leader Failure Recovery

---

2. **Paxos Implementation: - Paxos Mode A (Crash Fault Tolerance):-**

Paxos Mode A is designed to provide **Crash Fault Tolerance** in a distributed system. It ensures that all nodes agree on a single value even if some nodes crash or become temporarily unavailable. The algorithm achieves consensus through four phases: **Prepare, Promise, Accept, and Accepted**. Consensus is reached when a majority of nodes agree on the proposed value.

1. **Prepare:-**In the Prepare phase, a proposer initiates the consensus process by generating a unique proposal number and sending a **Prepare Request** to all acceptor nodes.
2. **Promise:-** After receiving a Prepare Request, each acceptor checks the proposal number.
   If the proposal number is greater than any previously seen proposal number, the acceptor sends a **Promise Message**.
3. **Accept:-** Once the proposer receives promises from a majority of acceptors, it sends an Accept Request containing the proposal number and the value to be agreed upon.
4. **Accepted:-** Acceptors verify that the proposal number matches their promise. If valid, they accept the value and send an Accepted Message.

❖ **create file inside src folder**:- touch paxos.py

```python
transaction = "TX001 : Pay Rs 100"

print("\nClient Sending Transaction...")
print(f"Node 5 received transaction: {transaction}")

print("\nPHASE 1 : PREPARE\n")

for i in range(1, 5):
    print(f"Leader -> Node {i} : PREPARE")

print("\nPHASE 2 : PROMISE\n")

for i in range(1, 5):
    print(f"Node {i} -> PROMISE")

print("\nPHASE 3 : ACCEPT\n")

for i in range(1, 5):
    print(
        f"Leader -> Node {i} : ACCEPT ({transaction})"
    )

print("\nPHASE 4 : ACCEPTED\n")

for i in range(1, 5):
    print(f"Node {i} -> ACCEPTED")

print("\nMajority Reached")
print("Transaction Committed")
```

- The paxos.py code is a **simulation of the Paxos consensus algorithm**. It demonstrates how a transaction is agreed upon by multiple nodes using the **four Paxos phases: Prepare, Promise, Accept, and Accepted**.

❖ **Run command**:-python paxos.py This output shows a **successful Paxos consensus execution** for a transaction.

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/distributed-consensus-engine/src$ python paxos.py

Client Sending Transaction...
Node 5 received transaction: TX001 : Pay Rs 100

PHASE 1 : PREPARE

Leader -> Node 1 : PREPARE
Leader -> Node 2 : PREPARE
Leader -> Node 3 : PREPARE
Leader -> Node 4 : PREPARE

PHASE 2 : PROMISE

Node 1 -> PROMISE
Node 2 -> PROMISE
Node 3 -> PROMISE
Node 4 -> PROMISE

PHASE 3 : ACCEPT

Leader -> Node 1 : ACCEPT (TX001 : Pay Rs 100)
Leader -> Node 2 : ACCEPT (TX001 : Pay Rs 100)
Leader -> Node 3 : ACCEPT (TX001 : Pay Rs 100)
Leader -> Node 4 : ACCEPT (TX001 : Pay Rs 100)

PHASE 4 : ACCEPTED

Node 1 -> ACCEPTED
Node 2 -> ACCEPTED
Node 3 -> ACCEPTED
Node 4 -> ACCEPTED

Majority Reached
Transaction Committed
```

Prepare, Promise Accept and Accepted Phase

---

3. **PBFT Implementation Mode B (Byzantine-Fault Tolerance): -**

Practical Byzantine Fault Tolerance (PBFT) is a consensus algorithm designed to allow distributed systems to continue operating correctly even when some nodes behave maliciously, send incorrect information, or fail unexpectedly. Unlike Paxos, which mainly handles crash failures, PBFT can tolerate Byzantine faults, where nodes may act arbitrarily or maliciously. Mode B implementing the four PBFT phases:

1. **Pre-Prepare:-** The consensus process begins when a client sends a transaction request to the primary (leader) node. The primary assigns a sequence number to the request and broadcasts a **PRE-PREPARE** message to all replica nodes.
2. **Prepare:**- After receiving the PRE-PREPARE message, each replica verifies the request. If the request is valid, the replica broadcasts a **PREPARE** message to all other replicas
3. **Commit:**-Once a node collects the required number of matching PREPARE messages, it broadcasts a **COMMIT** message to all replicas.

➢ **PBFT Implementation:-**

❖ **create a file name pbft.py inside src folder**:- touch pbft.py

pbft.py

src > pbft.py > run_pbft

```
# PBFT Simulation

NUM_NODES = 5
PRIMARY = 5

transaction = "TX002 : Pay Rs 500"

def run_pbft():

    print("\n====== PBFT PROTOCOL ======")

    print("\nClient Sending Transaction...")
    print(f"Primary Node {PRIMARY} received transaction: {transaction}")

    # Phase 1
    print("\nPHASE 1 : PRE-PREPARE\n")

    for node in range(1, NUM_NODES):
        print(
            f"Primary -> Node {node} : PRE-PREPARE ({transaction})"
        )

    # Phase 2
    print("\nPHASE 2 : PREPARE\n")

    prepare_votes = 0

    for node in range(1, NUM_NODES):
        print(f"Node {node} -> PREPARE")
        prepare_votes += 1

    print(
        f"\nPrepare Votes Received = {prepare_votes}"
    )

    # Phase 3
    print("\nPHASE 3 : COMMIT\n")

    commit_votes = 0

    for node in range(1, NUM_NODES):
        print(f"Node {node} -> COMMIT")
        commit_votes += 1

    print(
        f"\nCommit Votes Received = {commit_votes}"
    )

    if commit_votes >= 4:
        print("\nPBFT CONSENSUS REACHED")
        print("Transaction Committed")
    else:
        print("\nPBFT CONSENSUS FAILED")


if __name__ == "__main__":
    ▷ Generate Mermaid Diagram | 💬 Open Chat @mermaid-chart
    run_pbft()
```

- The pbft.py code is a **simulation of the PBFT (Practical Byzantine Fault Tolerance) protocol**. It demonstrates how distributed nodes reach consensus through the **Pre-Prepare, Prepare, and Commit** phases.

❖ **Run command:-**python pbft.py When a majority of nodes agree, consensus is achieved and the transaction is committed.

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/Assignment/distributed-consensus-engine/src$ python pbft.py

===== PBFT PROTOCOL =====

Client Sending Transaction...
Primary Node 5 received transaction: TX002 : Pay Rs 500

PHASE 1 : PRE-PREPARE

Primary -> Node 1 : PRE-PREPARE (TX002 : Pay Rs 500)
Primary -> Node 2 : PRE-PREPARE (TX002 : Pay Rs 500)
Primary -> Node 3 : PRE-PREPARE (TX002 : Pay Rs 500)
Primary -> Node 4 : PRE-PREPARE (TX002 : Pay Rs 500)

PHASE 2 : PREPARE

Node 1 -> PREPARE
Node 2 -> PREPARE
Node 3 -> PREPARE
Node 4 -> PREPARE

Prepare Votes Received = 4

PHASE 3 : COMMIT

Node 1 -> COMMIT
Node 2 -> COMMIT
Node 3 -> COMMIT
Node 4 -> COMMIT

Commit Votes Received = 4

PBFT CONSENSUS REACHED
Transaction Committed
```

Pre-Prepare, Prepare and Commit Phase

Note: I have created one merged file for all the three algorithms into node_consolidated .py, and below is the screenshots of the output for your reference:-

- **create file inside src folder**:-touch node_consolidated.py
- **run command:-** python node_consolidated.py

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/Assignment/distributed-consensus-engine/src$ python node_consolidated.py

=== INITIAL LEADER ELECTION ===

=== Cluster Status ===
Node 1 -> Follower (Alive)
Node 2 -> Follower (Alive)
Node 3 -> Follower (Alive)
Node 4 -> Follower (Alive)
Node 5 -> Leader (Alive)
[Heartbeat] Leader Node 5
[Heartbeat] Leader Node 5

============================
      PAXOS PROTOCOL
============================

Client Sending Transaction...
Leader Node 5 received transaction: TX001 : Pay Rs 100

PHASE 1 : PREPARE

Leader -> Node 1 : PREPARE
Leader -> Node 2 : PREPARE
Leader -> Node 3 : PREPARE
Leader -> Node 4 : PREPARE

PHASE 2 : PROMISE

Node 1 -> PROMISE
Node 2 -> PROMISE
Node 3 -> PROMISE
Node 4 -> PROMISE

PHASE 3 : ACCEPT

Leader -> Node 1 : ACCEPT (TX001 : Pay Rs 100)
Leader -> Node 2 : ACCEPT (TX001 : Pay Rs 100)
Leader -> Node 3 : ACCEPT (TX001 : Pay Rs 100)
Leader -> Node 4 : ACCEPT (TX001 : Pay Rs 100)

PHASE 4 : ACCEPTED

Node 1 -> ACCEPTED
Node 2 -> ACCEPTED
Node 3 -> ACCEPTED
Node 4 -> ACCEPTED

PAXOS CONSENSUS REACHED
Transaction Committed
[Heartbeat] Leader Node 5
```

```
==============================
         PBFT PROTOCOL
==============================

Client Sending Transaction...
Primary Node 5 received transaction: TX002 : Pay Rs 500

PHASE 1 : PRE-PREPARE

Primary -> Node 1 : PRE-PREPARE (TX002 : Pay Rs 500)
Primary -> Node 2 : PRE-PREPARE (TX002 : Pay Rs 500)
Primary -> Node 3 : PRE-PREPARE (TX002 : Pay Rs 500)
Primary -> Node 4 : PRE-PREPARE (TX002 : Pay Rs 500)

PHASE 2 : PREPARE

Node 1 -> PREPARE
Node 2 -> PREPARE
Node 3 -> PREPARE
Node 4 -> PREPARE

Prepare Votes Received = 4

PHASE 3 : COMMIT

Node 1 -> COMMIT
Node 2 -> COMMIT
Node 3 -> COMMIT
Node 4 -> COMMIT

Commit Votes Received = 4

PBFT CONSENSUS REACHED
Transaction Committed
[Heartbeat] Leader Node 5
[Heartbeat] Leader Node 5

***** Simulating Leader Crash *****

[ELECTION] New Leader Elected -> Node 4
[Heartbeat] Leader Node 4
[Heartbeat] Leader Node 4
```

```
============================
       PAXOS PROTOCOL
============================

Client Sending Transaction...
Leader Node 4 received transaction: TX003 : Pay Rs 200

PHASE 1 : PREPARE

Leader -> Node 1 : PREPARE
Leader -> Node 2 : PREPARE
Leader -> Node 3 : PREPARE

PHASE 2 : PROMISE

Node 1 -> PROMISE
Node 2 -> PROMISE
Node 3 -> PROMISE

PHASE 3 : ACCEPT

Leader -> Node 1 : ACCEPT (TX003 : Pay Rs 200)
Leader -> Node 2 : ACCEPT (TX003 : Pay Rs 200)
Leader -> Node 3 : ACCEPT (TX003 : Pay Rs 200)

PHASE 4 : ACCEPTED

Node 1 -> ACCEPTED
Node 2 -> ACCEPTED
Node 3 -> ACCEPTED

PAXOS CONSENSUS REACHED
Transaction Committed
[Heartbeat] Leader Node 4
[Heartbeat] Leader Node 4

============================
        PBFT PROTOCOL
============================

Client Sending Transaction...
Primary Node 4 received transaction: TX004 : Pay Rs 700

PHASE 1 : PRE-PREPARE

Primary -> Node 1 : PRE-PREPARE (TX004 : Pay Rs 700)
Primary -> Node 2 : PRE-PREPARE (TX004 : Pay Rs 700)
Primary -> Node 3 : PRE-PREPARE (TX004 : Pay Rs 700)

PHASE 2 : PREPARE

Node 1 -> PREPARE
Node 2 -> PREPARE
Node 3 -> PREPARE

Prepare Votes Received = 3

PHASE 3 : COMMIT

Node 1 -> COMMIT
Node 2 -> COMMIT
Node 3 -> COMMIT

Commit Votes Received = 3

PBFT CONSENSUS FAILED

===== SYSTEM EXECUTION COMPLETED =====
```

❖ **create a folder name:-** mkdir Byzantine_faults

➢ **Client Transaction Processing: -**

❖ **create a client.py file in Byzantine_faults folder:-** touch client.py

client.py

src > byzantine_faults > client.py > ...

```python
import time

count = 1

while True:

    print(
        f"Client Transaction TX{count}"
    )

    count += 1

    ▷ Generate Mermaid Diagram | Open Chat @mermaid-chart
    time.sleep(2)
```

- This client.py is acting as a **transaction generator (client node)** in your distributed consensus system. It Imports the time module so the program can pause between transactions.

❖ **Run command:-**python client.py

```
(base) hg1053@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/distributed-consensus-engine/src/byzantine_faults$ python client.py
Client Transaction TX1
Client Transaction TX2
Client Transaction TX3
Client Transaction TX4
Client Transaction TX5
```

Client Transaction

- **Cryptographic Key Distribution and Signature Verification:-**

  To ensure secure communication among distributed nodes, each node is assigned a unique public-private key pair. The private key remains confidential to the node, while the public key is shared with other nodes participating in the network. This process is known as key distribution and enables nodes to verify the authenticity of received messages.

- **create a file inside Byzantine_faults folder :-**touch crypto_utils.py

```python
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding


def generate_keys():

    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    public_key = private_key.public_key()

    return private_key, public_key


def sign_message(private_key, message):

    signature = private_key.sign(
        message.encode(),
        padding.PKCS1v15(),
        hashes.SHA256()
    )

    return signature


def verify_signature(
    public_key,
    message,
    signature
):

    try:

        public_key.verify(
            signature,
            message.encode(),
            padding.PKCS1v15(),
            hashes.SHA256()
        )

        return True

    except Exception:

        return False
```

- **create a file inside Byzantine_faults folder :-**touch test_crypto.py

```python
from crypto_utils import sign_message, verify_message
message = "TX001 : Pay Rs 100"
signature = sign_message(message)
print("Message:", message)
print("Signature Generated")
if verify_message(message, signature):
    print("Signature Verified")
else:
    print("Verification Failed")
```

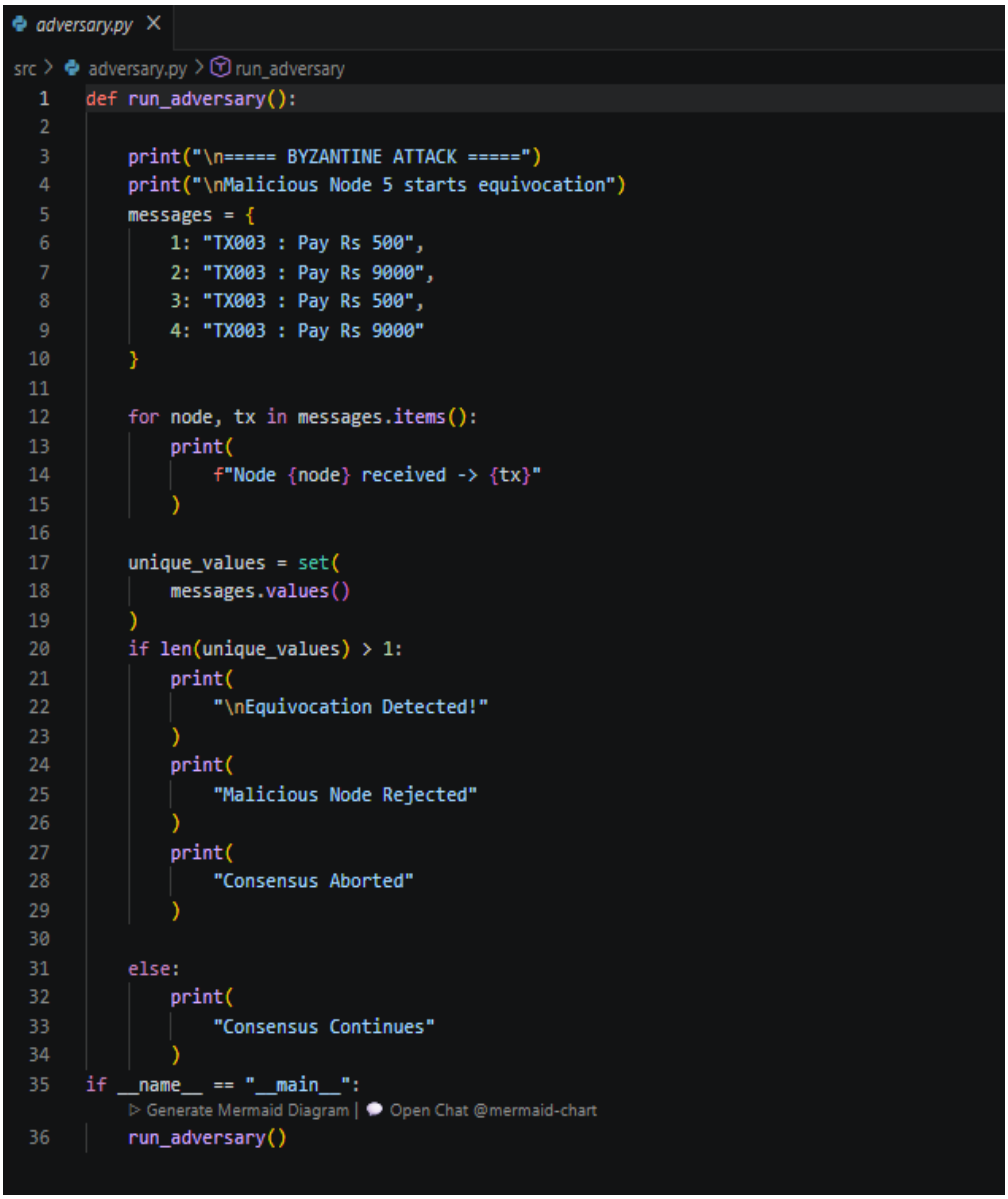
- **Run command** :- python test_crypto.py

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/Assignment/distributed-consensus-engine/src/byzantine_faults$ python test_crypto.py
Message: TX001 : Pay Rs 100
Signature Generated
Signature Verified
```

4. **Byzantine Adversary Node:-**

A **Byzantine Adversary Node** is a malicious or faulty node in a distributed system that does not follow the protocol correctly. Unlike crash failures, where a node simply stops working, a Byzantine node may send incorrect, conflicting, or misleading messages to different nodes. Such behaviour can cause inconsistencies and make consensus difficult to achieve.

- ❖ **create a file name adversary.py**:- touch adversary.py



```python
def run_adversary():

    print("\n===== BYZANTINE ATTACK =====")
    print("\nMalicious Node 5 starts equivocation")
    messages = {
        1: "TX003 : Pay Rs 500",
        2: "TX003 : Pay Rs 9000",
        3: "TX003 : Pay Rs 500",
        4: "TX003 : Pay Rs 9000"
    }

    for node, tx in messages.items():
        print(
            f"Node {node} received -> {tx}"
        )

    unique_values = set(
        messages.values()
    )
    if len(unique_values) > 1:
        print(
            "\nEquivocation Detected!"
        )
        print(
            "Malicious Node Rejected"
        )
        print(
            "Consensus Aborted"
        )

    else:
        print(
            "Consensus Continues"
        )
if __name__ == "__main__":
    run_adversary()
```

- This code simulates a **Byzantine Attack** in a PBFT system. The malicious node sends **different transaction values to different nodes**, a behavior known as **equivocation**. The purpose is to test whether the consensus protocol can detect and reject dishonest nodes.

- **Run command:-** python adversary.py (This output demonstrates a **Byzantine Equivocation Attack** in your PBFT implementation.)

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/distributed-consensus-engine/src$ python adversary.py

===== BYZANTINE ATTACK =====

Malicious Node 5 starts equivocation
Node 1 received -> TX003 : Pay Rs 500
Node 2 received -> TX003 : Pay Rs 9000
Node 3 received -> TX003 : Pay Rs 500
Node 4 received -> TX003 : Pay Rs 9000

Equivocation Detected!
Malicious Node Rejected
Consensus Aborted
```

PBFT Byzantine Equivocation Attack

- **create a file named pbft_demo.py and write the code in the file**:- touch pbft_demo.py

pbft_demo.py

```
print("PBFT MODE")
print("==========")

transaction = "TX001 : Pay Rs 100"
# PRE-PREPARE
print("\nPRE-PREPARE PHASE")
print("---------------")
print(f"\nPrimary Node -> {transaction}")
# PREPARE
print("\nPREPARE PHASE")
print("-------------")

for i in range(1, 6):
    print(f"\nNode {i} -> PREPARE")
# BYZANTINE ATTACK
print("\n\nBYZANTINE ATTACK")
print("----------------")
print("\nAdversary Node 99 sent conflicting transaction")
print("Signature Verification Failed")
print("Ignoring Malicious Node")
# COMMIT
print("\n\nCOMMIT PHASE")
print("------------")

for i in range(1, 6):
    print(f"\nNode {i} -> COMMIT")

# RESULT
print("\n\nPBFT RESULT")
print("-----------")

print("\nTransaction COMMITTED")
```

- The code is a **simple demonstration of PBFT (Practical Byzantine Fault Tolerance)**. It shows how a distributed system can **detect a malicious node, ignore its invalid transaction, and still reach consensus**.

❖ **Run command:-** python pbft_demo.py(this output demonstrates that PBFT detects and ignores malicious activity before committing the transaction.)

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/distributed-consensus-engine/src$ python pbft_demo.py
PBFT MODE
==========

PRE-PREPARE PHASE
---------------

Primary Node -> TX001 : Pay Rs 100

PREPARE PHASE
------------

Node 1 -> PREPARE

Node 2 -> PREPARE

Node 3 -> PREPARE

Node 4 -> PREPARE

Node 5 -> PREPARE


BYZANTINE ATTACK
----------------

Adversary Node 99 sent conflicting transaction
Signature Verification Failed
Ignoring Malicious Node


COMMIT PHASE
------------

Node 1 -> COMMIT

Node 2 -> COMMIT

Node 3 -> COMMIT

Node 4 -> COMMIT

Node 5 -> COMMIT


PBFT RESULT
-----------

Transaction COMMITTED
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/distributed-consensus-eng
```

PBFT Byzantine Fault Handling

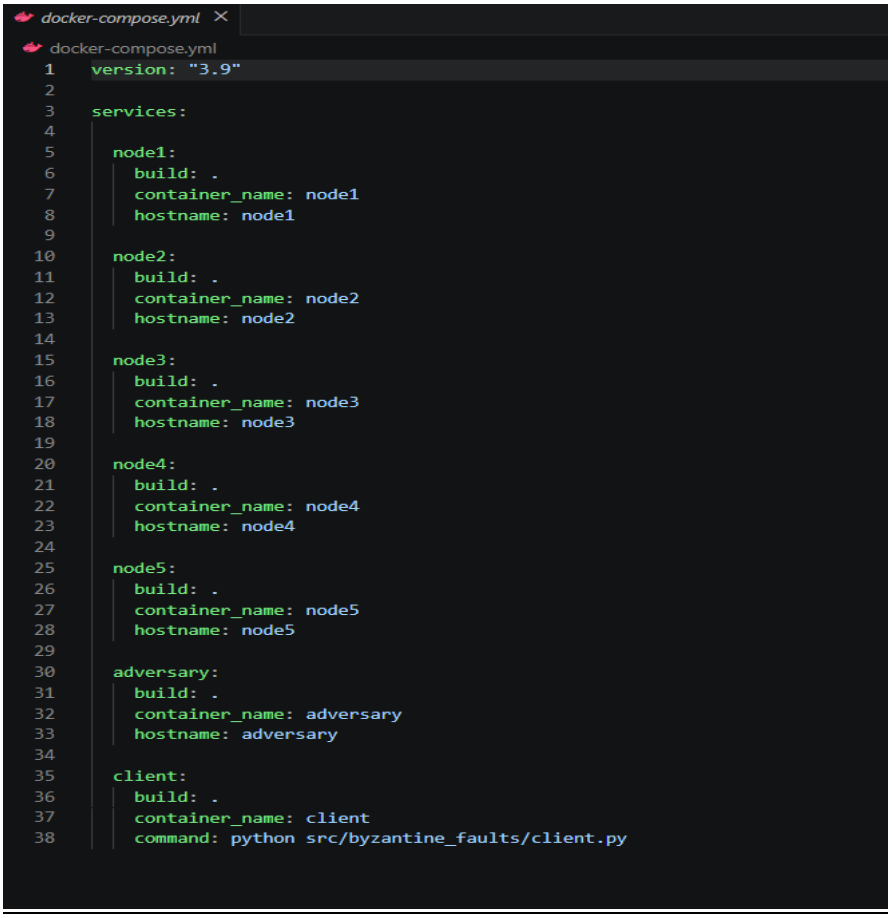
---

5. **Docker Networking & Chaos Testing:-**

   The purpose of **Docker Networking and Chaos Testing** is to evaluate the fault tolerance of the distributed consensus system under adverse conditions. A Docker-based environment is created consisting of five consensus nodes, one Byzantine node, and a network simulator. Faults such as node failures, network delays, and temporary network partitions are intentionally introduced to verify that leader election and consensus mechanisms continue to function correctly.

➢ **Docker Networking:-**

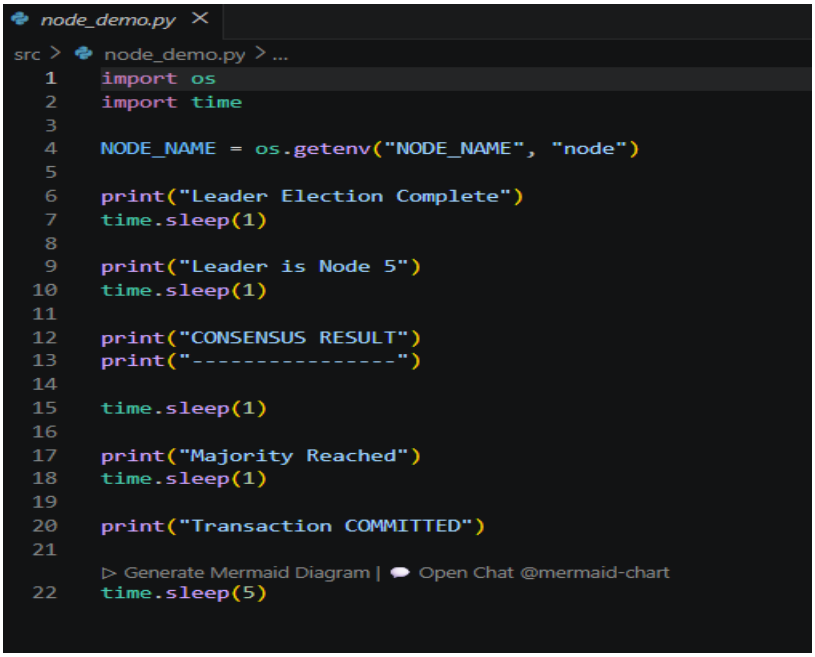
❖ **Create a docker-compose.yml file** :- touch docker compose.yml



```yaml
version: "3.9"

services:

  node1:
    build: .
    container_name: node1
    hostname: node1

  node2:
    build: .
    container_name: node2
    hostname: node2

  node3:
    build: .
    container_name: node3
    hostname: node3

  node4:
    build: .
    container_name: node4
    hostname: node4

  node5:
    build: .
    container_name: node5
    hostname: node5

  adversary:
    build: .
    container_name: adversary
    hostname: adversary

  client:
    build: .
    container_name: client
    command: python src/byzantine_faults/client.py
```

- It is defining **multiple Docker containers** that will run your distributed consensus system.

❖ **create file:-**touch node_demo.py



```python
import os
import time

NODE_NAME = os.getenv("NODE_NAME", "node")

print("Leader Election Complete")
time.sleep(1)

print("Leader is Node 5")
time.sleep(1)

print("CONSENSUS RESULT")
print("----------------")

time.sleep(1)

print("Majority Reached")
time.sleep(1)

print("Transaction COMMITTED")

time.sleep(5)
```

- The code Simulates that leader election has finished and Node 5 became leader.

❖ **create docker file** :- touch Dockerfile
This Dockerfile is the recipe Docker uses to create an image

```
Dockerfile
1   FROM python:3.11
2
3   WORKDIR /app
4
5   COPY . .
6
7   RUN pip install -r requirements.txt
8
9   CMD ["python", "src/node_demo.py"]
```

❖ **Run command**:-docker-compose up –build (This output shows that Docker successfully started all your containers)

```
Attaching to adversary, client, node1, node2, node3, node4, node5
node5  | Leader Election Complete
node5  | Leader is Node 5
node5  | CONSENSUS RESULT
node5  | ---------------
node5  | Majority Reached
node5  | Transaction COMMITTED
node4  | Leader Election Complete
node4  | Leader is Node 5
node4  | CONSENSUS RESULT
node4  | ---------------
node4  | Majority Reached
node4  | Transaction COMMITTED
node1  | Leader Election Complete
node1  | Leader is Node 5
node1  | CONSENSUS RESULT
node1  | ---------------
node1  | Majority Reached
node1  | Transaction COMMITTED
node3  | Leader Election Complete
node3  | Leader is Node 5
node3  | CONSENSUS RESULT
node3  | ---------------
node3  | Majority Reached
node3  | Transaction COMMITTED
node2  | Leader Election Complete
node2  | Leader is Node 5
node2  | CONSENSUS RESULT
node2  | ---------------
node2  | Majority Reached
node2  | Transaction COMMITTED
adversary  | Leader Election Complete
adversary  | Leader is Node 5
adversary  | CONSENSUS RESULT
adversary  | ---------------
adversary  | Majority Reached
adversary  | Transaction COMMITTED
```

Docker-Based Consensus Commit

- **Chaos Testing:-** it is a technique used to intentionally introduce failures into a distributed system to verify that it can continue operating correctly under adverse conditions**.**
- **Create a test file:-** mkdir tests
- **Then create a chaos_test :-** touch tests/chaos_test.sh

```
$ chaos_test.sh ×
src > tests > $ chaos_test.sh
1   #!/bin/bash
2
3   echo "================================"
4   echo "CHAOS TEST STARTED"
5   echo "================================"
6
7   echo "Stopping Leader Node"
8   docker stop node1
9
10  sleep 10
11
12  echo "Checking Running Containers"
13  docker ps
14
15  echo "Restarting Leader"
16  docker start node1
17
18  sleep 5
19
20  echo "CHAOS TEST COMPLETED"
    ▷ Generate Mermaid Diagram | 💬 Open Chat @mermaid-chart
21
```

- **Run command:-** chmod +x tests/chaos_test.sh
  - ./tests/chaos_test.sh

This output shows that **Chaos Test script** is working, because our node containers are staying alive and the script successfully stopped and restarted the leader container.

```
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/distributed-consensus-engine/src$ ./tests/chaos_test.sh
================================
CHAOS TEST STARTED
================================
Stopping Leader Node
node1
Checking Running Containers
CONTAINER ID   IMAGE                                COMMAND                  CREATED         STATUS         PORTS                                                NAMES
3b68a85d884b   distributed-consensus-engine-node5   "python src/node.py"     8 minutes ago   Up 8 minutes                                                        node5
eb179eba3dd7   distributed-consensus-engine-node3   "python src/node.py"     8 minutes ago   Up 8 minutes                                                        node3
2a21542e30c6   distributed-consensus-engine-adversary   "python src/node.py"     8 minutes ago   Up 8 minutes                                                        adversary
7204e3ea85d9   distributed-consensus-engine-node2   "python src/node.py"     8 minutes ago   Up 8 minutes                                                        node2
8d96a48f5aee   distributed-consensus-engine-client   "python src/byzantin…"   8 minutes ago   Up 8 minutes                                                        client
cea06ead2a72   distributed-consensus-engine-node4   "python src/node.py"     8 minutes ago   Up 8 minutes                                                        node4
fa53cac639eb   mongo:latest                         "docker-entrypoint.s…"   3 weeks ago     Up 9 hours     0.0.0.0:8080->27017/tcp, [::]:8080->27017/tcp   mongoDB
Restarting Leader
node1
CHAOS TEST COMPLETED
(base) hg1058@LAPTOP-HG:/mnt/e/IIT JODHPUR/Trimester 3 subjects/Fundamentals of Distributed System/distributed-consensus-engine/src$
```

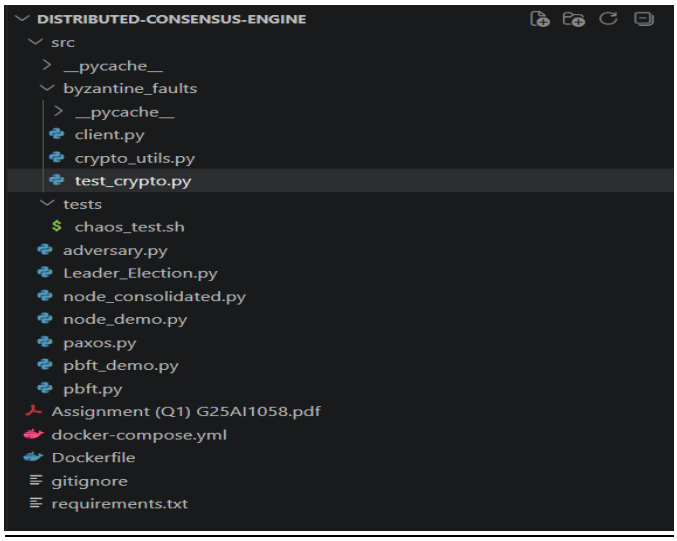
Chaos Testing Results

Folder and File Structure:

- This is my structure of folder and files as I have first individually run the leader election, paxos and pbft then run in consolidated node.

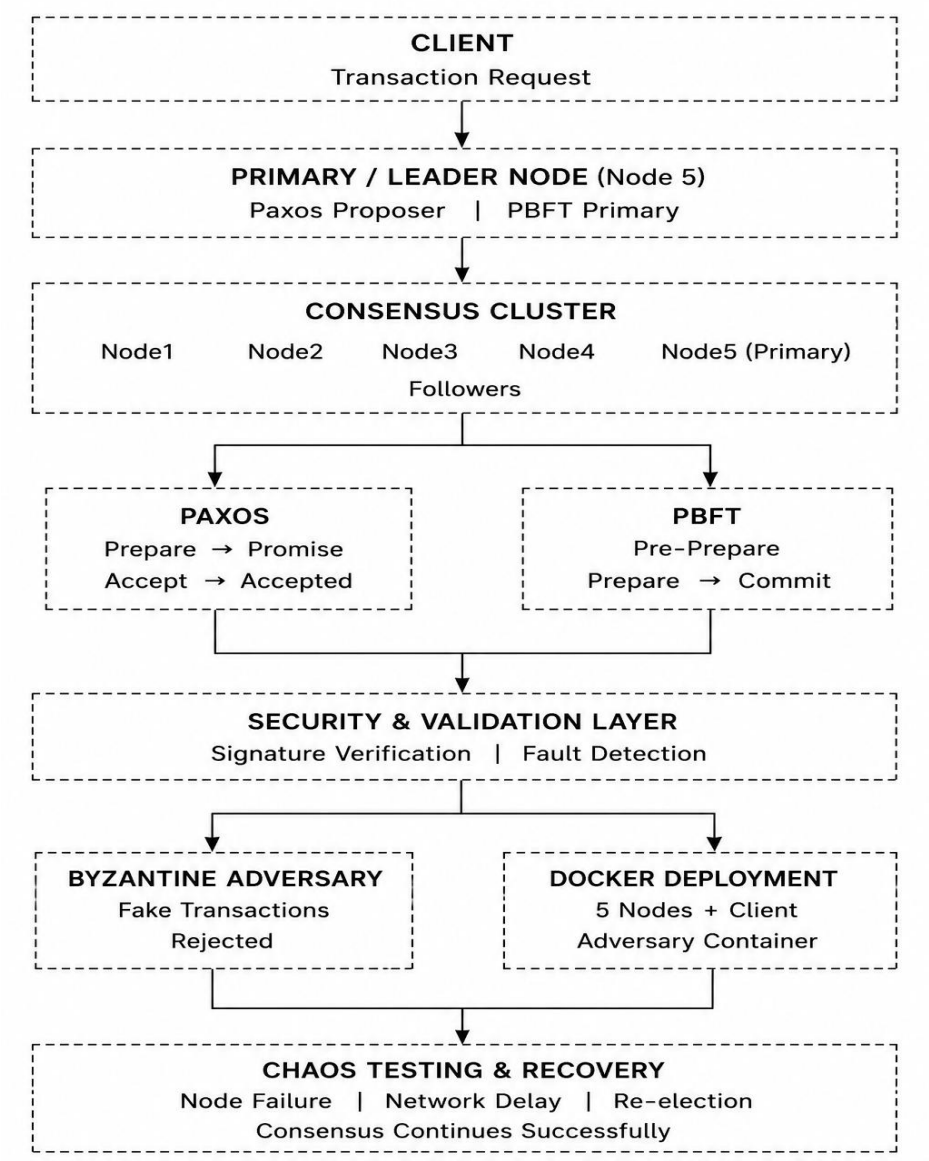
distributed-consensus-engine/

```
--src/
    ├── Leader_Election.py
    ├── paxos.py
    ├── pbft.py
    ├── node_demo.py
    ├── pbft_demo.py
    ├── node_consolidated.py (t leader election, paxos and pbft)
    ├── adversary.py
    │
    └── byzantine_faults/
        ├── client.py
        └── crypto_utils.py
        └── test_crypto.py
── tests/
    └── chaos_test.sh

── docker-compose
── Dockerfile
── requirements.txt
── gitignore
└── Assignment_Report.pdf
```



Structure of folder and files created in VScode

Architecture Design



Distributed Consensus Engine Architecture using Paxos and PBFT

Video Link:-

https://drive.google.com/file/d/1Af12rjOGNeAUekPXiH7ZyzwUkeA12K8C/view?usp=drive_link

---